

Lab 1:RISC-V引导与裸机启动

[WHU OS Lab 仓库](#)

系统设计部分

在实验一中，主要完成了几个重要的模块：

- `entry.S`:C语言环境设置
- `start.c`:在M-mode初始化
- `main.c` 初始化工作，目前初始串口和输出模块
- `spinlock`：自旋锁
- `uart`：串口驱动程序
- `print`：打印输出

架构设计说明

实验一的架构为，qemu将PC设置为0x80000000，进入函数 `entry`，该函数给每个CPU核心创建了一个运行栈，提供了C语言的运行环境，随后跳转到函数 `start`。`start` 函数是为了将M模式过渡到S模式，并且跳转到 `main`。`main` 目前只是检测是否启动成功，设置了 `printf` 函数来查看操作系统是否成功启动。`printf` 目前用于串口 `uart` 输出,同时需要自旋锁 `spinlock` 实现互斥。

与 xv6 对比分析

尽量在xv6下简化，在 `start` 函数中，没有进行计数器的初始化，在 `print` 中，简化采取了串口 `uart` 输出，没有使用控制台 `console`，在 `main` 函数中，只是自行做了多核的检测。

设计决策理由

因为实验刚刚起步的阶段，功能还比较简单，所以目的在尽力简化xv6，实现最简单的启动。

实验过程部分

先完成 `entry.S` 的编写，然后写 `start.c`，再写自旋锁和串口驱动，完成 `printf` 函数，最后编写 `main`。随后进行调试，首先发现 `common.mk` 中，编译的工具链出现问题，修改了部分代码

```
# 允许外部覆盖前缀；默认用裸机工具链
```

```
CROSS_COMPILE ?= riscv64-unknown-elf- #重点在这里，使用riscv64-unknown-elf-
```

```
TOOLPREFIX := $(CROSS_COMPILE)

override CC      := $(TOOLPREFIX)gcc
override LD      := $(TOOLPREFIX)ld
override OBJCOPY := $(TOOLPREFIX)objcopy
override OBJDUMP := $(TOOLPREFIX)objdump

export CROSS_COMPILE CC LD OBJCOPY OBJDUMP
```

随后发现，`start` 函数无法跳转到 `main` 函数，研究发现，缺少代码

```
w_pmpaddr0(0x3fffffffffffffff);
w_pmpcfg0(0xf);
```

这部分代码负责允许S模式进行物理内存访问，才能写入 `main` 的地址。

实现步骤记录

在周一下午的课程完成了基本代码的书写，但一直苦于调试，在周二周三都在学习使用gdb调试，也是在这段时间，发现了 `common.mk` 文件的错误，和内存访问代码的缺失。周三基本跑通代码，周五撰写实验报告。

问题与解决方案

1. 对 CMake 不熟悉，不懂得如何进行构建，所以在 `common.mk` 发生错误时，很久不知道怎么调整，最后只能在大模型的帮助下，才找出正确的解决方案
2. 对 gdb 不熟悉,在网络中进行了搜索才知道很多技巧，比如如何查看寄存器的值。
3. 对于 RISC-V 的 S-mode 和 M-mode 的代码不熟悉，所在在 `start` 函数中，不知道各个汇编代码的意思，查看了手册才更清晰

源码理解总结

总结一些关键代码：

1. `csrr a1, mhartid` 表示 把 CSR（控制与状态寄存器）的值读到通用寄存器 `rd` 中。
`mhartid` 是一个 只读 CSR，编号 `0xF14`。它记录 当前硬件线程（hart）的 ID 这句代码出现在 `entry.S` 中，用于给每个CPU分配一个栈
2. `start` 函数

```
#include "riscv.h"
void main();

__attribute__((aligned(16))) uint8 CPU_stack[4096 * NCPU];

void start()
{

    unsigned long x = r_mstatus(); //读取mstatus寄存器

    x &= ~MSTATUS_MPP_MASK; //清空mpp字段

    x |= MSTATUS_MPP_S; //将MPP字段设置为 'S'

    w_mstatus(x); //重新写回

    w_mepc((uint64)main); //写入main，帮助跳转到main函数

    //关闭分页

    w_satp(0);

    //中断陷入Supervisor模式

    w_medeleg(0xffff);

    w_mideleg(0xffff);

    w_sie(r_sie() | SIE_SEIE | SIE_STIE);
```

```

// 允许Supervisor访问物理内存

w_pmpaddr0(0x3fffffffffffffff);

w_pmpcfg0(0xf);


//获取硬件ID

int id = r_mhartid();

//写入tp

w_tp(id);


asm volatile("mret");
}

```

3. 源码从 entry.S -> start.c -> main.c , 其中main调用了printf, 涉及锁和串口调用

测试验证部分

```

#include "riscv.h"

#include "lib/print.h"

#include "proc/proc.h"

#include "dev/uart.h"

volatile static int started = 0;


int main()
{

    if(mycpuid()==0)

    {

```

```
    uart_init();

    print_init();

    printf("\n");

    printf("xv6 kernel is booting\n");

    printf("\n");

    started = 1;
}

if(mycpuid()==1)
{
    uart_init();

    print_init();

    printf("\n");

    printf("cpu1 is booting\n");

    printf("\n");
}

if(mycpuid()==2)
{
    uart_init();

    print_init();

    printf("\n");

    printf("cpu2 is booting\n");

    printf("\n");
}
```

```
    if(mycpuid()==3)
    {
        uart_init();

        print_init();

        printf("\n");

        printf("cpu3 is booting\n");

        printf("\n");
    }

    while (1);
}
```

测试三个核心是否启动

功能测试结果

```
cpu2 is booting
cpu1 is booting

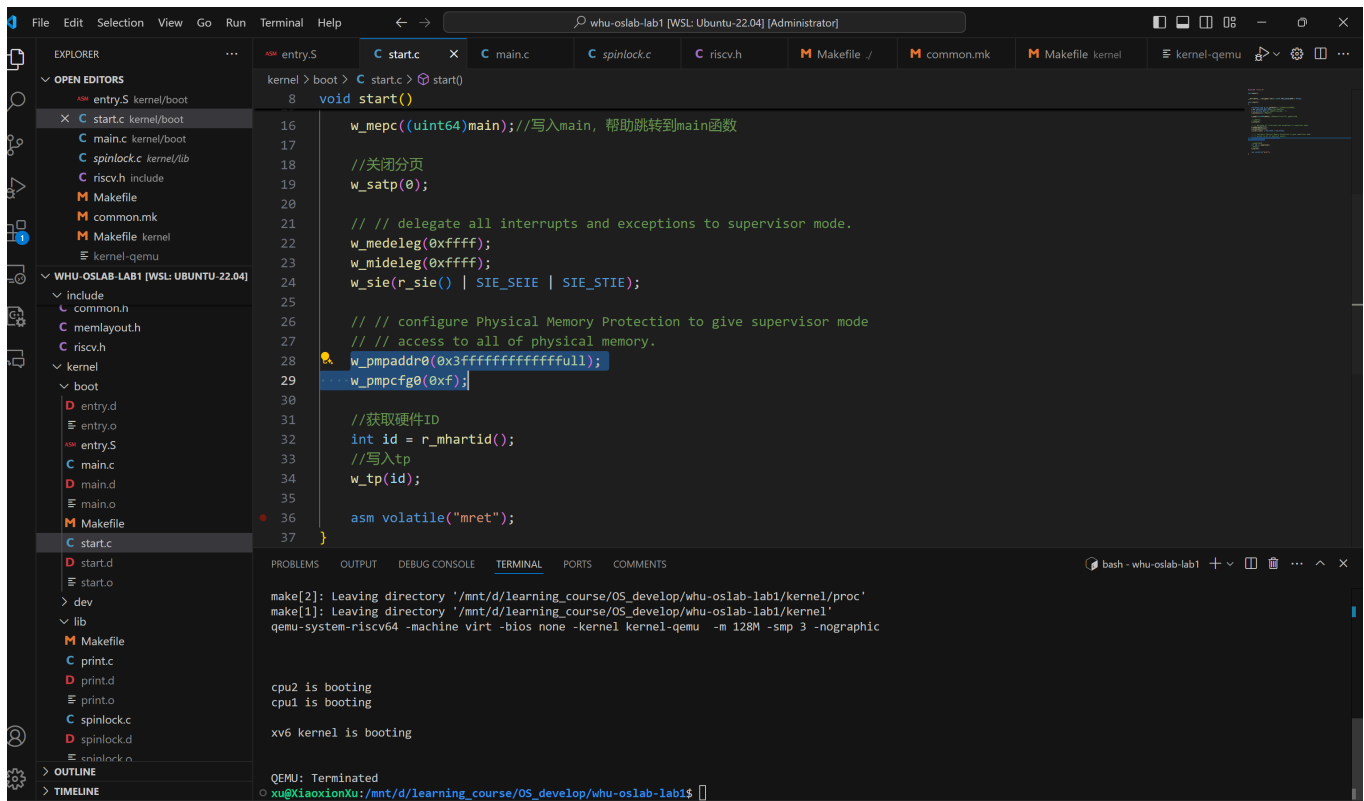
xv6 kernel is booting
```

表明测试成功

异常测试

未见异常

运行截图/录屏



额外任务

并行计算

方法一：每一次sum++加锁

```
#include "riscv.h"

#include "lib/print.h"

#include "proc/proc.h"

#include "dev/uart.h"

#include "lib/lock.h"

volatile static int started = 0;

volatile static int sum = 0;

static struct spinlock sum_lock;
```

```
int main()

{
    int cpuid = r_tp();

    if(cpuid == 0) {

        print_init();

        spinlock_init(&sum_lock, "sum_lock");

        printf("cpu %d is booting!\n", cpuid);

        __sync_synchronize();

        started = 1;

        for(int i = 0; i < 1000; i++)//注意把循环次数改成了1000
        {

            spinlock_acquire(&sum_lock);//上锁

            sum++;

            spinlock_release(&sum_lock);//释放

        }

        printf("cpu %d report: sum = %d\n", cpuid, sum);
    } else {

        while(started == 0);

        __sync_synchronize();

        printf("cpu %d is booting!\n", cpuid);

        for(int i = 0; i < 1000; i++)
        {

            spinlock_acquire(&sum_lock);//上锁

            sum++;
```



```

        spinlock_release(&sum_lock); //释放

    }

    printf("cpu %d report: sum = %d\n", cpuid, sum);

}

while (1);

}

```

最后结果

```

cpu 0 is booting!
cpu 1 is booting!
cpu 0 report: sum = 1871
cpu 1 report: sum = 2000

```

将1000000改成1000的原因是，自旋锁性能过差，必须减少数据量才能快速出结果，如果是1000000，恐怕要几十分钟。**锁粒度太细**

方法二：在for循环外边上锁

为了不赘余，展示关键的代码

```

    spinlock_acquire(&sum_lock);
    for(int i = 0; i < 1000000; i++)
    {
        sum++;
    }
    spinlock_release(&sum_lock);

```

终端输出

```

cpu 0 report: sum = 1000000
cpu 1 report: sum = 2000000

```

这种上锁方式性能很好，为粗粒度上锁，问题是串行执行

并行输出

去掉了print的锁，两个核心分别打印大小的写的26个字母，终端输出乱序字母，每一次不一样

Aa BbCc DdEe F GgHh IiJj KkLl MmNn OoPp QqRr SsTt UuVv WwXx YyZz=